

Sprint 2 Structured Notes: Values, Data Types, Variables, and Type Checks

0. What Sprint 2 Is About

Sprint 2 is about learning how JavaScript stores and understands data.

In Sprint 1, you learned how JavaScript can make a web page respond to actions, like a button click.

In Sprint 2, you learn what data JavaScript works with when those actions happen.

For example:

Button clicked → what value changed?
Form submitted → what text did the user type?
Score updated → is the score really a number?
Task added → where is the task stored?

The main goal of Sprint 2 is:

Store data safely and understand what kind of value you are working with.

By the end of Sprint 2, you should understand:

- values
- data types
- variables
- `let`
- `const`
- initialization
- reassignment
- variable naming
- dynamic typing
- `typeof`
- the `typeof null` bug
- primitive values
- object references
- why `const` objects can still be changed inside
- how to check the type of a value

1. The Core Idea of Sprint 2

JavaScript programs work with **values**.

A value is a piece of data.

Examples:

```
25
"hello"
true
null
{ name: "Ada" }
```

Each value has a type.

For example:

```
25           // number
"hello"      // string
true        // boolean
null        // null value
{ name: "Ada" } // object
```

A JavaScript program often does three things with values:

```
Store values.
Read values.
Change values.
```

To store a value, you usually use a variable.

Example:

```
let age = 25;
```

Simple meaning:

```
Create a variable called age.
Store the value 25 in it.
```

2. What Is a Value?

A **value** is actual data that JavaScript can use.

Examples of values:

```
100
"JavaScript"
false
undefined
null
{ active: true }
```

A value can be:

- a number
- text
- true or false
- empty
- missing
- an object
- a special advanced type like `Symbol` or `BigInt`

You can print values with `console.log()`:

```
console.log(100);
console.log("JavaScript");
console.log(true);
```

Expected output:

```
100
JavaScript
true
```

3. What Is a Data Type?

A **data type** tells JavaScript what kind of value it has.

Simple meaning:

A data type is the category of a value.

Example:

```
25      // number
"hello" // string
true    // boolean
```

Why data types matter:

JavaScript behaves differently depending on the type of value.

Example:

```
console.log(5 + 10);    // 15
console.log("5" + 10);  // "510"
```

In the first line, `5` is a number.

In the second line, `"5"` is a string.

So JavaScript treats the result differently.

4. The Main Data Types in Sprint 2

Sprint 2 covers these data types:

```
Number
String
Boolean
undefined
null
Object
Symbol
BigInt
```

You do not need to master every advanced detail yet.

For now, you need to recognize what each type means.

Part A: Data Types

5. Number

A **Number** is used for numeric values.

Examples:

```
25  
3.14  
-10  
0
```

Numbers are used for math.

Example:

```
let score = 10;  
let bonus = 5;  
  
console.log(score + bonus);
```

Expected output:

```
15
```

More examples:

```
let price = 19.99;  
let quantity = 3;  
let total = price * quantity;  
  
console.log(total);
```

Expected output:

```
59.97
```

Use numbers for:

- age
- score
- price
- quantity
- total
- temperature
- count
- page number

Example:

```
let age = 16;  
let temperature = -5;  
let pageNumber = 1;
```

6. String

A **String** is text.

Strings are written inside quotes.

Examples:

```
"hello"  
'hello'  
`hello`
```

All three are strings.

Example:

```
let name = "Ada";  
let course = "JavaScript";  
let message = "Welcome";  
  
console.log(name);  
console.log(course);  
console.log(message);
```

Expected output:

```
Ada  
JavaScript  
Welcome
```

Use strings for:

- names
- messages
- usernames
- emails
- titles
- button text
- input text

Example:

```
let username = "sangeeth";  
let email = "student@example.com";  
let buttonText = "Submit";
```

Important:

```
25      // number  
"25"    // string
```

They look similar, but they are not the same type.

7. Boolean

A **Boolean** is either:

```
true  
false
```

A boolean means yes/no or on/off.

Examples:

```
let isLoggedIn = true;
let isAdmin = false;
let hasFinishedSprint1 = true;
```

Use booleans for:

- active or inactive
- logged in or not logged in
- valid or invalid
- completed or not completed
- visible or hidden
- enabled or disabled

Example:

```
let taskCompleted = false;

console.log(taskCompleted);
```

Expected output:

```
false
```

Simple meaning:

```
true means yes.
false means no.
```

8. undefined

`undefined` usually means:

```
A variable exists, but no value has been assigned yet.
```

Example:


```
let age;  
  
console.log(age);
```

Expected output:

```
undefined
```

The variable `age` exists, but it has no value yet.

Later, you can assign a value:

```
let age;  
  
console.log(age); // undefined  
  
age = 25;  
  
console.log(age); // 25
```

Simple meaning:

```
undefined means JavaScript does not have a value there yet.
```

Common beginner situation:

```
let username;  
  
console.log(username);
```

Output:

```
undefined
```

This does not mean the program is always broken.

It means the variable was declared but not given a value.

9. null

`null` means:

Intentionally empty.

Example:

```
let selectedUser = null;
```

Simple meaning:

There is no selected user right now.
That is intentional.

Another example:

```
let currentTask = null;
```

This means:

There is no current task selected.

undefined vs null

`undefined` usually means no value was assigned yet.

`null` usually means the programmer intentionally set the value to nothing.

Example:

```
let firstName;  
let middleName = null;  
  
console.log(firstName);  
console.log(middleName);
```

Expected output:

```
undefined  
null
```

Simple comparison:

```
undefined → no value assigned yet  
null      → intentionally empty
```

10. Object

An **Object** stores related data together.

An object uses key-value pairs.

Example:

```
const user = {  
  name: "Ada",  
  age: 25,  
  active: true  
};
```

This object has three properties:

```
name   → "Ada"  
age    → 25  
active → true
```

You can read object properties with dot notation:

```
console.log(user.name);  
console.log(user.age);  
console.log(user.active);
```

Expected output:

```
Ada  
25  
true
```

Objects are useful when several pieces of data belong together.

Example:

```
const player = {  
  username: "coder123",  
  score: 50,  
  online: true  
};
```

This is better than having disconnected variables:

```
let username = "coder123";  
let score = 50;  
let online = true;
```

Objects help keep related data organized.

11. Symbol

A **Symbol** is a special unique value.

Example:

```
const id = Symbol("id");  
  
console.log(typeof id);
```

Expected output:

```
symbol
```

Beginner meaning:

A Symbol creates a unique identifier.

Even if two symbols use the same description, they are different.

Example:

```
const firstId = Symbol("id");  
const secondId = Symbol("id");  
  
console.log(firstId === secondId);
```

Expected output:

false

Why?

Because every Symbol is unique.

Beginner note:

You only need basic awareness of Symbol right now.
You will not use it often in beginner projects.

12. BigInt

A **BigInt** is used for very large integers.

A BigInt ends with `n`.

Example:

```
const bigNumber = 123456789012345678901234567890n;  
  
console.log(typeof bigNumber);
```

Expected output:

```
bigint
```

Beginner meaning:

```
BigInt is for whole numbers that are too large for normal Number values.
```

Important:

```
100    // number
100n    // bigint
```

They are different types.

Beginner note:

```
You only need basic awareness of BigInt right now.
Most beginner projects use normal Number values.
```

Part B: Variables

13. What Is a Variable?

A **variable** is a name used to store a value.

Example:

```
let score = 10;
```

Simple meaning:

```
Create a variable called score.
Put the value 10 in it.
```

You can use the variable later:

```
let score = 10;  
  
console.log(score);
```

Expected output:

```
10
```

Without variables, you would have to repeat values again and again.

Variables help you:

- store data
- reuse data
- update data
- give meaning to values

Compare this:

```
console.log(10 + 5);
```

With this:

```
let score = 10;  
let bonus = 5;  
  
console.log(score + bonus);
```

The second version is easier to understand.

14. Parts of a Variable Statement

Example:

```
let score = 10;
```

Parts:

```
let    → keyword
score → variable name
=      → assignment operator
10     → value
;      → end of statement
```

Simple reading:

```
Let score be 10.
```

Another example:

```
const username = "Ada";
```

Parts:

```
const    → keyword
username → variable name
=        → assignment operator
"Ada"    → value
;        → end of statement
```

15. Declaration

A **declaration** creates a variable name.

Example:

```
let age;
```

This declares a variable called `age`.

At this point, it does not have a real value yet.

Example:


```
let age;  
  
console.log(age);
```

Expected output:

```
undefined
```

Simple meaning:

The variable exists, but it has not been given a value.

16. Initialization

Initialization means giving a variable its first value.

Example:

```
let age = 25;
```

Here, `age` is initialized with `25`.

Another example:

```
let username = "Ada";
```

Here, `username` is initialized with `"Ada"`.

Declaration without initialization:

```
let score;
```

Declaration with initialization:

```
let score = 10;
```

Simple meaning:

Initialization is the first assignment.

17. Assignment

Assignment means putting a value into a variable.

The assignment operator is:

=

Example:

```
let score;  
score = 10;
```

This means:

Create score.
Then assign 10 to score.

Important:

=

Does not mean “equal” in JavaScript assignment.

It means:

Put the value on the right into the variable on the left.

Example:

```
let total = 100;
```

Simple reading:

Put 100 into total.

18. Reassignment

Reassignment means giving a variable a new value after it already had one.

Example:

```
let score = 10;  
score = 15;  
  
console.log(score);
```

Expected output:

15

The value changed from 10 to 15.

Full example:

```
let score;  
  
console.log(score); // undefined  
  
score = 10;  
  
console.log(score); // 10  
  
score = 20;  
  
console.log(score); // 20
```

What happens:

1. score is declared.
2. score starts as undefined.
3. score is assigned 10.
4. score is reassigned 20.

Part C: let and const

19. let

Use `let` when the value may change.

Example:

```
let score = 10;

score = 15;

console.log(score);
```

Expected output:

```
15
```

This works because `let` allows reassignment.

Good examples for `let`:

```
let score = 0;
let count = 1;
let currentPage = 1;
let userInput = "";
let total = 0;
```

These values may change as the program runs.

Example:

```
let count = 0;

count = count + 1;
count = count + 1;

console.log(count);
```

Expected output:

2

Use `let` when the value needs to update.

20. const

Use `const` when the variable should not be reassigned.

Example:

```
const appName = "Study Tracker";  
  
console.log(appName);
```

Expected output:

Study Tracker

This is not allowed:

```
const points = 5;  
points = 8;
```

This causes an error because `const` variables cannot be reassigned.

Simple meaning:

`const` means the variable name cannot be pointed to a different value later.

Good examples for `const`:

```
const maxScore = 100;  
const appName = "Study Tracker";  
const birthYear = 2008;
```

Use `const` when the variable should keep the same value.

21. Beginner Rule: Use const First

A useful beginner rule is:

```
Use const first.  
Use let only when reassignment is needed.
```

Example:

```
const username = "Ada";  
let score = 0;  
  
score = score + 10;
```

Why?

```
username does not need to change.  
score is expected to change.
```

This habit makes your code safer.

If a value should not change, `const` protects it from accidental reassignment.

22. Debug Challenge: const Reassignment Error

Broken code:

```
const points = 5;  
points = 8;
```

Problem:

```
points was declared with const.  
const does not allow reassignment.
```

So JavaScript throws an error.

Correct version using `let`:

```
let points = 5;
points = 8;

console.log(points);
```

Expected output:

8

Correct version using `const` without reassignment:

```
const points = 5;

console.log(points);
```

Expected output:

5

23. const with Objects

This is one of the most important beginner details.

A `const` object can still have its properties changed.

Example:

```
const player = {
  name: "Ada",
  active: true
};

player.active = false;

console.log(player);
```

Expected output:

```
{ name: "Ada", active: false }
```

This works.

Why?

Because `const` prevents the variable from being reassigned.

It does not freeze the inside of the object.

This is allowed:

```
const player = { name: "Ada" };  
  
player.name = "Grace";  
  
console.log(player.name);
```

Expected output:

```
Grace
```

This is not allowed:

```
const player = { name: "Ada" };  
  
player = { name: "Grace" };
```

Why?

Because this tries to make `player` point to a completely different object.

Simple rule:

```
const protects the variable name from reassignment.  
const does not make object properties unchangeable.
```

Mental model:


```
const player → points to one object
```

You cannot make player point to a different object.
But you can change data inside the object.

Part D: Naming Variables

24. Why Variable Names Matter

Variable names should describe the value.

Weak names:

```
let x = 25;  
let y = "Ada";  
let z = true;
```

Better names:

```
let age = 25;  
let username = "Ada";  
let isActive = true;
```

Good names make code easier to read.

When possible, use names that explain the purpose of the value.

Examples:

```
let score = 90;  
let totalPrice = 49.99;  
let isLoggedIn = false;  
let selectedCourse = null;
```

25. Case Sensitivity

JavaScript variable names are case-sensitive.

That means uppercase and lowercase letters matter.

Example:

```
let score = 10;  
let Score = 20;  
  
console.log(score);  
console.log(Score);
```

Expected output:

```
10  
20
```

`score` and `Score` are different variables.

These are also different:

```
let username = "Ada";  
let userName = "Grace";  
let UserName = "Linus";
```

Beginner warning:

Be consistent with spelling and capitalization.

26. camelCase

JavaScript commonly uses `camelCase` for variable names.

Examples:

```
let userName = "Ada";  
let totalPrice = 100;
```

```
let currentPage = 1;  
let isLoggedIn = true;
```

In camelCase:

The first word starts lowercase.
Each next word starts with a capital letter.

Example:

```
user name → userName  
total price → totalPrice  
is logged in → isLoggedIn
```

Good beginner habit:

Use clear camelCase names for variables.

27. Boolean Naming Pattern

Boolean variables often start with words like:

```
is  
has  
can  
should
```

Examples:

```
let isActive = true;  
let hasAccount = false;  
let canEdit = true;  
let shouldSave = false;
```

This makes the variable read like a yes/no question.

Example:

```
let isLoggedIn = true;
```

Question:

Is logged in?

Answer:

true

Part E: Dynamic Typing

28. What Is Dynamic Typing?

JavaScript is dynamically typed.

Simple meaning:

A variable can hold different types at different times.

Example:

```
let value = 10;  
  
console.log(typeof value);  
  
value = "ten";  
  
console.log(typeof value);
```

Expected output:

number
string

At first, `value` held a number.

Later, `value` held a string.

This is allowed in JavaScript.

29. Why Dynamic Typing Can Be Useful

Dynamic typing can be flexible.

Example:

```
let result;  
  
result = 100;  
console.log(result);  
  
result = "Complete";  
console.log(result);
```

This works because JavaScript allows the variable type to change.

But this flexibility can also create bugs.

30. Why Dynamic Typing Can Cause Bugs

Example:

```
let score = 10;  
score = "10";  
  
console.log(score + 5);
```

Expected output:

105

Why?

Because `score` is now a string.

So JavaScript joins the text instead of doing normal addition.

This is not the same as:

```
let score = 10;  
  
console.log(score + 5);
```

Expected output:

15

Beginner rule:

Do not casually change a variable from one type to another.
Check the type when something behaves strangely.

Part F: typeof

31. What Is typeof?

`typeof` tells you the type of a value.

Example:

```
console.log(typeof 10);  
console.log(typeof "hello");  
console.log(typeof true);
```

Expected output:

```
number  
string  
boolean
```

Simple meaning:

typeof asks JavaScript: what kind of value is this?

32. typeof Examples

```
console.log(typeof 25);  
console.log(typeof "JavaScript");  
console.log(typeof false);  
console.log(typeof undefined);  
console.log(typeof null);  
console.log(typeof { name: "Ada" });  
console.log(typeof Symbol("id"));  
console.log(typeof 100n);
```

Expected output:

```
number  
string  
boolean  
undefined  
object  
object  
symbol  
bigint
```

Important result:

```
console.log(typeof null);
```

Output:

```
object
```

This is a special JavaScript problem.

33. The typeof null Bug

In JavaScript:

```
console.log(typeof null);
```

Output:

```
object
```

This is confusing because `null` is not really an object.

It is a historical JavaScript bug.

Beginner rule:

```
Remember that typeof null returns "object".  
Do not let this confuse you.
```

Example:

```
let selectedUser = null;  
  
console.log(selectedUser);  
console.log(typeof selectedUser);
```

Expected output:

```
null  
object
```

Simple explanation:

```
The value is null.  
But typeof reports it as object because of an old JavaScript bug.
```

34. Using typeof for Debugging

`typeof` is useful when your code behaves strangely.

Example:


```
let userAge = "25";  
  
console.log(userAge);  
console.log(typeof userAge);
```

Expected output:

```
25  
string
```

This tells you that `userAge` looks like a number, but JavaScript sees it as text.

Another example:

```
let total = 100;  
  
console.log(typeof total);
```

Expected output:

```
number
```

Use `typeof` when you need to confirm what type of value you have.

35. Practice: typeReport Function

A useful Sprint 2 mini-build is a function that reports the type of a value.

Code:

```
function typeReport(value) {  
  return "The value is a " + typeof value;  
}  
  
console.log(typeReport("hello"));  
console.log(typeReport(42));  
console.log(typeReport(true));  
console.log(typeReport(null));
```

Expected output:

```
The value is a string
The value is a number
The value is a boolean
The value is a object
```

The last line sounds strange:

```
The value is a object
```

That happens because:

```
typeof null
```

returns:

```
object
```

You can improve the function later, but for Sprint 2, this version helps you practice `typeof`.

Part G: Primitive Values and Object References

36. Primitive vs Non-Primitive Values

JavaScript values can be grouped into two broad categories:

```
Primitive values
Non-primitive values
```

Primitive values include:

```
number
string
boolean
undefined
null
```

```
symbol  
bigint
```

Non-primitive values include:

```
objects  
arrays  
functions
```

In Sprint 2, the main non-primitive value you need to understand is the object.

37. Primitive Values Are Copied by Value

Primitive values are copied by value.

Example:

```
let a = 10;  
let b = a;  
  
a = 20;  
  
console.log(a);  
console.log(b);
```

Expected output:

```
20  
10
```

Why is `b` still `10`?

Because `b` got its own copy of the primitive value.

Mental model:

```
a has 10.  
b gets a copy of 10.
```

Then a changes to 20.
b stays 10.

Another example:

```
let firstName = "Ada";  
let secondName = firstName;  
  
firstName = "Grace";  
  
console.log(firstName);  
console.log(secondName);
```

Expected output:

Grace
Ada

`secondName` does not change because strings are primitive values.

38. Objects Are Copied by Reference

Objects behave differently.

Objects are copied by reference.

Example:

```
const user = { name: "Alice" };  
const sameUser = user;  
  
user.name = "Ada";  
  
console.log(sameUser.name);
```

Expected output:

Ada

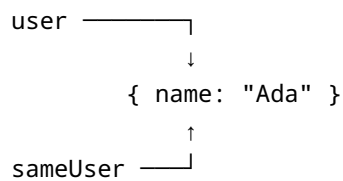
Why?

Because both variables point to the same object.

Mental model:

user points to the object.
sameUser points to the same object.
Changing the object through user also affects sameUser.

Diagram:



Another example:

```
const firstUser = { name: "Ada" };
const secondUser = firstUser;

secondUser.name = "Grace";

console.log(firstUser.name);
console.log(secondUser.name);
```

Expected output:

Grace
Grace

Both variables show `Grace` because both point to the same object.

39. Why Object References Matter

Object references matter because changing data through one variable can affect another variable.

Example:

```
const original = { score: 10 };  
const copy = original;  
  
copy.score = 99;  
  
console.log(original.score);
```

Expected output:

99

This can surprise beginners.

The variable named `copy` is not a true independent copy.

It is another reference to the same object.

Simple rule:

Primitive values copy the value.
Objects copy the reference.

This is one of the most important ideas in Sprint 2.

40. Primitive Copy vs Object Reference Comparison

Primitive example:

```
let firstScore = 90;  
let secondScore = firstScore;  
  
firstScore = 100;  
  
console.log(firstScore);  
console.log(secondScore);
```

Expected output:

```
100  
90
```

Object example:

```
const firstUser = { name: "Ada" };  
const secondUser = firstUser;  
  
firstUser.name = "Grace";  
  
console.log(firstUser.name);  
console.log(secondUser.name);
```

Expected output:

```
Grace  
Grace
```

Main difference:

```
Primitive values are copied independently.  
Objects can share the same reference.
```

Part H: Core Sprint 2 Code Pattern

41. Core Code Pattern

Type this code yourself:

```
let age;  
  
console.log(age); // undefined  
  
age = 25;  
  
console.log(age); // 25  
  
const user = { name: "Alice" };
```

```
const sameUser = user;  
  
user.name = "Ada";  
  
console.log(sameUser.name); // Ada
```

What this code teaches:

1. A declared variable without a value is undefined.
2. A let variable can be assigned later.
3. A const object can have its properties changed.
4. Objects are copied by reference.
5. user and sameUser point to the same object.

42. Step-by-Step Explanation of the Core Pattern

Line 1:

```
let age;
```

This creates the variable `age`.

No value is assigned yet.

So `age` is `undefined`.

Line 2:

```
console.log(age);
```

Output:

```
undefined
```

Line 3:

```
age = 25;
```

Now the value `25` is assigned to `age`.

Line 4:

```
console.log(age);
```

Output:

```
25
```

Line 5:

```
const user = { name: "Alice" };
```

This creates an object and stores a reference to it in `user`.

Line 6:

```
const sameUser = user;
```

This does not create a separate object.

It makes `sameUser` point to the same object as `user`.

Line 7:

```
user.name = "Ada";
```

This changes the `name` property inside the shared object.

Line 8:

```
console.log(sameUser.name);
```

Output:

```
Ada
```

Why?

Because `sameUser` points to the same object that was changed.

Part I: Sprint 2 Starter Practice

43. Starter Code

Type this code:

```
let score = 10;

const player = {
  name: "Ada",
  active: true
};

score = 15;
player.active = false;

console.log(score);
console.log(player);
console.log(typeof null);
```

Expected output:

```
15
{ name: "Ada", active: false }
object
```

What this teaches:

```
score changes because it was declared with let.
player.active changes because const does not freeze object properties.
typeof null returns object because of a JavaScript bug.
```

44. Starter Code Explanation

This line creates a changeable variable:

```
let score = 10;
```

This line creates a constant variable that points to an object:

```
const player = {  
  name: "Ada",  
  active: true  
};
```

This changes `score`:

```
score = 15;
```

Allowed because `score` uses `let`.

This changes a property inside the object:

```
player.active = false;
```

Allowed because the object itself is still the same object.

This prints the score:

```
console.log(score);
```

Output:

```
15
```

This prints the player object:

```
console.log(player);
```

Output:

```
{ name: "Ada", active: false }
```

This checks the type of `null`:

```
console.log(typeof null);
```

Output:

```
object
```

Part J: Common Sprint 2 Mistakes

45. Mistake 1: Reassigning const

Wrong:

```
const score = 10;  
score = 20;
```

Problem:

```
score was declared with const.  
const does not allow reassignment.
```

Fix:

```
let score = 10;  
score = 20;
```

Use `let` when the value must change.

46. Mistake 2: Thinking const Freezes Objects

Wrong idea:

```
A const object cannot be changed at all.
```

Correct idea:

A const object cannot be reassigned, but its properties can be changed.

Example:

```
const user = { name: "Ada" };  
  
user.name = "Grace";  
  
console.log(user.name);
```

Expected output:

Grace

This works because the object itself was not replaced.

47. Mistake 3: Confusing undefined and null

Wrong idea:

undefined and null mean exactly the same thing.

Correct idea:

undefined usually means no value was assigned.
null usually means intentionally empty.

Example:

```
let a;  
let b = null;  
  
console.log(a);  
console.log(b);
```

Expected output:

```
undefined  
null
```

48. Mistake 4: Forgetting typeof null

Wrong expectation:

```
console.log(typeof null);
```

Beginner may expect:

```
null
```

Actual output:

```
object
```

Correct understanding:

```
typeof null returns "object" because of a historical JavaScript bug.
```

49. Mistake 5: Changing a Variable's Type Accidentally

Example:

```
let score = 10;  
score = "10";  
  
console.log(score + 5);
```

Output:

```
105
```

Problem:

```
score changed from number to string.
```

Safer habit:

```
let score = 10;  
  
console.log(typeof score);
```

Output:

```
number
```

Use `typeof` when you are unsure.

50. Mistake 6: Thinking Object Copy Creates a Separate Object

Wrong idea:

```
const user = { name: "Ada" };  
const copy = user;
```

Beginner may think:

```
copy is a new independent object.
```

Correct idea:

```
copy points to the same object.
```

Example:

```
const user = { name: "Ada" };  
const copy = user;  
  
copy.name = "Grace";  
  
console.log(user.name);
```

Expected output:

Grace

Part K: Practice Tasks

51. Warm-Up Drill 1: let Variables

Task:

Create three `let` variables and reassign them.

Example:

```
let score = 10;
let level = 1;
let message = "Start";

score = 20;
level = 2;
message = "Continue";

console.log(score);
console.log(level);
console.log(message);
```

Expected output:

```
20
2
Continue
```

Self-check:

Can you explain why reassignment works here?

Answer:

Because the variables were declared with `let`.

52. Warm-Up Drill 2: const Variables

Task:

Create three `const` variables and do not reassign them.

Example:

```
const appName = "Study Tracker";
const maxScore = 100;
const isPublic = true;

console.log(appName);
console.log(maxScore);
console.log(isPublic);
```

Expected output:

```
Study Tracker
100
true
```

Self-check:

What would happen if you tried to reassign `maxScore`?

Answer:

JavaScript would throw an error because `maxScore` was declared with `const`.

53. Warm-Up Drill 3: Create Each Type

Task:

Create one value of each type covered in Sprint 2.

Example:

```
const numberValue = 25;
const stringValue = "hello";
const booleanValue = true;
let undefinedValue;
const nullValue = null;
const objectValue = { name: "Ada" };
const symbolValue = Symbol("id");
const bigintValue = 100n;
```

Now check each type:

```
console.log(typeof numberValue);
console.log(typeof stringValue);
console.log(typeof booleanValue);
console.log(typeof undefinedValue);
console.log(typeof nullValue);
console.log(typeof objectValue);
console.log(typeof symbolValue);
console.log(typeof bigintValue);
```

Expected output:

```
number
string
boolean
undefined
object
object
symbol
bigint
```

Important:

```
null gives object.
object gives object.
```

Do not confuse them.

54. Debug Challenge 1: const Reassignment

Broken code:

```
const points = 5;  
points = 8;
```

Task:

Explain why this throws an error.

Answer:

```
points was declared with const.  
const variables cannot be reassigned.
```

Fixed version:

```
let points = 5;  
points = 8;  
  
console.log(points);
```

Expected output:

```
8
```

55. Debug Challenge 2: Object Reference

Code:

```
const user = { name: "Ada" };  
const sameUser = user;  
  
sameUser.name = "Grace";  
  
console.log(user.name);
```

Task:

Predict the output.

Expected output:

Grace

Explanation:

user and sameUser point to the same object.
Changing sameUser.name changes the shared object.
user.name reads from that same object.

56. Mini-Build: typeReport(value)

Task:

Write a function that returns a sentence describing the value type.

Code:

```
function typeReport(value) {  
  return "The value is a " + typeof value;  
}  
  
console.log(typeReport("hello"));  
console.log(typeReport(42));  
console.log(typeReport(true));  
console.log(typeReport(undefined));  
console.log(typeReport(null));
```

Expected output:

```
The value is a string  
The value is a number  
The value is a boolean  
The value is a undefined  
The value is a object
```

Note:

The grammar is not perfect.
The purpose is to practice typeof.

57. Extra Challenge: Primitive Copy vs Object Reference

Task:

Show the difference between copying a primitive value and copying an object reference.

Primitive version:

```
let first = 10;
let second = first;

first = 99;

console.log(first);
console.log(second);
```

Expected output:

```
99
10
```

Object version:

```
const firstUser = { name: "Ada" };
const secondUser = firstUser;

firstUser.name = "Grace";

console.log(firstUser.name);
console.log(secondUser.name);
```

Expected output:

```
Grace
Grace
```

Explanation:

The primitive value was copied independently.
The object reference was shared.

Part L: Sprint 2 Mental Models

58. Mental Model: Variable as a Label

For beginners, think of a variable as a label for a value.

Example:

```
let age = 25;
```

Mental model:

```
age → 25
```

If reassigned:

```
age = 26;
```

Mental model:

```
age → 26
```

This works well for primitive values.

59. Mental Model: Object Reference

For objects, think of the variable as pointing to the object.

Example:

```
const user = { name: "Ada" };
```

Mental model:

```
user → { name: "Ada" }
```

If another variable gets the same object:

```
const sameUser = user;
```

Mental model:

```
user      ↘  
           ↓  
sameUser → { name: "Ada" }
```

Both variables point to the same object.

60. Mental Model: let vs const

```
let  → this variable may point to a different value later  
const → this variable should keep pointing to the same value
```

Example:

```
let score = 0;  
score = 10;
```

Allowed.

```
const score = 0;  
score = 10;
```

Not allowed.

For objects:

```
const user = { name: "Ada" };  
user.name = "Grace";
```

Allowed because the variable still points to the same object.

Part M: Key Vocabulary

61. Vocabulary

Value

A piece of data JavaScript can use.

Example:

```
25  
"hello"  
true
```

Data Type

The category of a value.

Example:

```
number, string, boolean
```

Variable

A name used to store a value.

Example:

```
let score = 10;
```

Declaration

Creating a variable name.

Example:

```
let age;
```

Initialization

Giving a variable its first value.

Example:

```
let age = 25;
```

Assignment

Putting a value into a variable.

Example:

```
age = 25;
```

Reassignment

Giving a variable a new value after it already had one.

Example:

```
age = 26;
```

let

A keyword for variables that can be reassigned.

Example:

```
let score = 0;  
score = 10;
```

const

A keyword for variables that cannot be reassigned.

Example:

```
const maxScore = 100;
```

Number

A numeric value.

Example:

```
25  
3.14  
-10
```

String

Text inside quotes.

Example:

```
"hello"
```

Boolean

A true or false value.

Example:

```
true  
false
```

undefined

A value meaning no value has been assigned yet.

Example:

```
let age;  
console.log(age); // undefined
```

null

A value meaning intentionally empty.

Example:

```
let selectedUser = null;
```

Object

A value that stores related data as properties.

Example:

```
const user = { name: "Ada", active: true };
```

Symbol

A unique special value.

Example:

```
const id = Symbol("id");
```

BigInt

A very large whole number.

Example:

```
const large = 100n;
```

Dynamic Typing

The JavaScript behavior that allows a variable to hold different types at different times.

Example:

```
let value = 10;  
value = "ten";
```

typeof

An operator that reports the type of a value.

Example:

```
console.log(typeof "hello"); // string
```

Primitive Value

A simple value copied by value.

Examples:

```
number, string, boolean, undefined, null, symbol, bigint
```

Reference

A connection or pointer to an object.

Example:

```
const sameUser = user;
```

Part N: Revision Questions

62. Sprint 2 Revision Questions

Try to answer these without looking at the notes.

1. What is a value?
2. What is a data type?
3. What is a variable?
4. What is the difference between declaration and initialization?
5. What is reassignment?
6. When should you use `let`?
7. When should you use `const`?
8. Can a `const` variable be reassigned?
9. Can a property inside a `const` object be changed?

10. What is a Number?
 11. What is a String?
 12. What is a Boolean?
 13. What does `undefined` mean?
 14. What does `null` mean?
 15. What is the difference between `undefined` and `null`?
 16. What does `typeof` do?
 17. What does `typeof null` return?
 18. Why is `typeof null` confusing?
 19. What does dynamic typing mean?
 20. Why can dynamic typing cause bugs?
 21. What is a primitive value?
 22. What does “copied by value” mean?
 23. What does “copied by reference” mean?
 24. Why does changing an object through one variable affect another variable?
 25. What is the most important difference between primitive values and objects?
-

Part O: Self-Check Answers

63. Self-Check Answers

1. What is a value?

A value is a piece of data JavaScript can use.

Examples:

```
25
"hello"
true
```

2. What is a data type?

A data type is the category of a value.

Examples:

```
number, string, boolean
```

3. What is a variable?

A variable is a name used to store a value.

Example:

```
let score = 10;
```

4. What is declaration?

Declaration creates a variable name.

Example:

```
let age;
```

5. What is initialization?

Initialization gives a variable its first value.

Example:

```
let age = 25;
```

6. What is reassignment?

Reassignment gives a variable a new value later.

Example:

```
let age = 25;  
age = 26;
```

7. When should you use let?

Use `let` when the variable needs to be reassigned.

Example:

```
let score = 0;  
score = 10;
```

8. When should you use const?

Use `const` when the variable should not be reassigned.

Example:

```
const maxScore = 100;
```

9. What does undefined mean?

`undefined` usually means a variable exists but has not been assigned a value.

Example:

```
let name;  
console.log(name); // undefined
```

10. What does null mean?

`null` means intentionally empty.

Example:

```
let selectedUser = null;
```

11. What does typeof do?

`typeof` reports the type of a value.

Example:

```
console.log(typeof 10); // number
```

12. What does typeof null return?

It returns:

object

13. Why does changing an object through one variable affect another variable?

Because both variables may point to the same object.

Example:

```
const user = { name: "Ada" };
const sameUser = user;

sameUser.name = "Grace";

console.log(user.name); // Grace
```

14. What is the difference between primitive copy and object reference?

Primitive values are copied independently.

Objects can share the same reference.

Part P: Final Sprint 2 Checklist

64. Final Checklist

You are ready to move on from Sprint 2 when you can do these things:

- [] Explain what a value is.
- [] Explain what a data type is.
- [] Identify Number, String, Boolean, undefined, null, Object, Symbol, and BigInt.
- [] Declare variables with let.
- [] Declare variables with const.
- [] Explain when to use let.
- [] Explain when to use const.
- [] Explain declaration.
- [] Explain initialization.
- [] Explain reassignment.
- [] Use clear camelCase variable names.
- [] Explain that JavaScript variable names are case-sensitive.

- [] Explain dynamic typing.
- [] Use typeof to check a value type.
- [] Remember that typeof null returns "object".
- [] Explain the difference between undefined and null.
- [] Explain primitive values.
- [] Explain that primitive values are copied by value.
- [] Explain that objects are copied by reference.
- [] Explain why changing one object reference can affect another variable.
- [] Explain why const does not freeze object properties.
- [] Debug a const reassignment error.
- [] Build a simple typeReport(value) function.

Part Q: One-Sentence Summary

65. Sprint 2 One-Sentence Summary

Sprint 2 teaches you how JavaScript stores data with variables, how values have types, how `let` and `const` work, how to check types with `typeof`, and why primitive values copy differently from objects.

Part R: Minimum Passing Standard

66. Minimum Passing Standard for Sprint 2

You understand Sprint 2 if you can clearly explain this:

Primitive values copy independently, but objects can share the same reference.

Example:

```
let a = 10;
let b = a;

a = 20;

console.log(b); // 10
```

Primitive value: independent copy.

```
const user = { name: "Ada" };  
const sameUser = user;  
  
sameUser.name = "Grace";  
  
console.log(user.name); // Grace
```

Object value: shared reference.

This is the most important conceptual jump in Sprint 2.